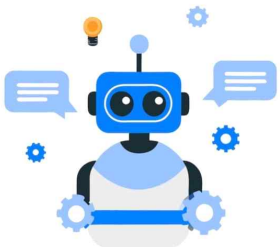
파이썬 고급과정

데이터 시각화

Choi Jae Seong
eins2017@naver.com



Chapter 01

자료구조(Data Structure) 데이터를 표현하고 관리하고 처리하기 위한 구조

자료구조의 목적?

- 수많은 자료(data)의 조직화를 통한 효율적인 저장, 관리
- 효율적인 메모리 사용

다양한 자료구조가 존재하지만, 모든 목적에 적합한 만능의 단일 자료구조는 없습니다. 삽입, 삭제, 비교, 교환, 검색 등의 자료 특성을 고려해 가장 효율적인 자료구조를 선택해야 하는 것이죠.

Collection 자료형 숫자형, 문자열, Boolean을 묶어서 처리하고 관리하기 위한 자료형

대표적으로 List, Tuple, Dictionary, Set가 있습니다.

List	순서가 있다	인덱싱 및 슬라이싱 가능	수정 가능	
Tuple	순서가 있다	인덱싱 및 슬라이싱 가능	수정 불가능	
Dictionary	순서가 없다	-	수정 가능	python의 Hash Table
Set	순서가 없다	-	수정 가능	Dictionary 형태에서 key 값만을 가진 자료형으로 집합연산이 가능함

List 변경 가능(mutable), 순서 존재(iterable), list 안에 객체들 중복 가능

다음의 코드를 실행해봅시다.

```
arr = []                          # 리스트 선언
arr2 = list()                     # 리스트 선언

arr.append(1)                     # 1 값 추가 가장 오른쪽에 넣는 것
arr.insert(2, 2)                  # index 2에 2 값 추가

arr.remove(1)                     # 값이 1인 것 삭제
del arr[1]                        # index 1에 있는 값 삭제

arr.pop()                         # 가장 왼쪽에 있는 값 삭제 및 return

arr.extend(arr2)                  # 리스트와 리스트 합치기
arr += arr2                       # 리스트와 리스트 합치기

arr3 = arr[1:6]                   # 리스트 slicing, index 1 ~ 5까지 복사

arr.sort()                        # 리스트 정렬 오름차순
arr.sort(reverse= True)           # 리스트 정렬 내림차순
sorted(arr)                       # 리스트 정렬 후 반환

print(" ".join(map(str,arr)))     # 출력

len(arr)                          # arr 길이(요소 개수)
max(arr)                          # arr에 있는 값 중 최대값
min(arr)                          # arr에 있는 값 중 최소값

arr.index(value)                  # value의 인덱스 반환
arr.count(value)                  # value의 개수 반환
```

*" ".join(map(str, arr))은 리스트 arr의 모든 요소들을 공백으로 구분하여 하나의 문자열로 결합하는 기능을 합니다

List를 String으로 - join

join은 기본적으로 **'구분자'.join(리스트)** 형태를 가지고 있습니다.

```
arr1 = ['a', 'b', 'c', 'd']
print(', '.join(arr1))

출력 결과: a, b, c, d
```

```
str1 = ''.join(arr1)
print(str1)

출력결과: abcd
```

```
a = ['is', 'you', 'down']
print('-'.join(a))
print(' '.join(a))

출력결과:
is-you-down
is you down
```

join 함수는 기본적으로 문자열로 이루어진 리스트만 가능하므로, int 리스트는 map 함수를 활용합니다.

```
arr2 = [1, 2, 3, 4]
print(f"<{(', '.join(map(str, arr2)))}>")

출력결과:
<1, 2, 3, 4>
```

Set 변경 가능(mutable), 순서 없음, 중복 불가능

다음의 코드를 실행해봅시다.

```
set1 = {1, 2, 3, 4}                # 집합선언
set2 = {2, 4, 6, 8}
set3 = set([1, 3, 5, 7])           # 집합선언

set3.add(9)                        # 집합 값 추가
set3.update({11})                  # 집합 더하기 |= 와 동일
set3.remove(11)                    # 집합 요소 삭제

set1 |= set2                       # 합집합
set1 &= set2                       # 교집합
set1 -= set2                       # 차집합
set1 ^= set2                       # 대칭차집합(합집합-교집합)

set1 <= set2                       # 집합 포함관계 확인
```

Tuple 삭제 및 변경 불가능(immutable), 순서 있음, 중복 가능, list와 유사함

다음의 코드를 실행해봅시다.

```
tuple1 = (1, 2, 3)                 # 튜플 선언
tuple2 = tuple(['a', 'b'])         # 튜플 선언2 리스트 to 튜플

len(tuple1)                        # 튜플 요수 개수
tuple1[1]                          # 튜플 인덱싱
tuple2[1:]                         # 튜플 슬라이싱
```

Dicionary 변경 가능(mutable), 순서 없음, key-value로 이루어져 있음, key는 immutable 객체만 가능하고 중복 불가능

다음의 코드를 실행해봅시다.

```
dic1 = {'a':1, 'b':2}          # 딕셔너리 선언

dic1['c'] = 3                  # 값 추가
del dic1['c']                  #key가 c인 요소 삭제

dic1.keys()                    # 키 목록
dic1.values()                  # 값 목록
dic1.items()                   # key - value 목록
dic1.clear()                   # 모든 요소 삭제
dic1.get('a')                  # key가 a인 value 반환
```